

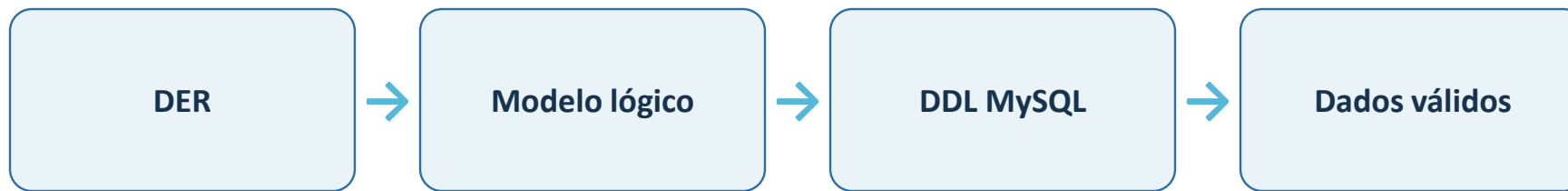
T05

Modelo físico e integridade no MySQL

Do esquema lógico a estruturas que rejeitam dados inválidos

Schemas • Tipos • Constraints • Integridade referencial • Anomalias

O modelo lógico define o quê; o físico decide como o MySQL protegerá



Ao final da T05, você deverá ser capaz de...

Selecionar tipos

Escolher tipos coerentes com domínio, precisão, tamanho e operações.

Declarar constraints

Traduzir PK, FK, unicidade, nulidade, padrões e checks em DDL.

Prever efeitos

Justificar ON DELETE/UPDATE e diagnosticar redundâncias e anomalias.

A aula está organizada em três blocos

1ª aula

Schemas e tipos: INT, DECIMAL, CHAR, VARCHAR, DATE, DATETIME, BOOLEAN e TEXT.

2ª aula

PRIMARY KEY, FOREIGN KEY, UNIQUE, NOT NULL, DEFAULT, CHECK e AUTO_INCREMENT.

3ª aula

Integridade, ações referenciais, redundâncias e anomalias.

Modelo físico não é apenas SQL: é a política de validade do banco

Estrutura

Schemas, tabelas, colunas, tipos e mecanismo de armazenamento.

Regras

Chaves, nulidade, unicidade, padrões e predicados de validação.

Comportamento

O que ocorre ao inserir, atualizar ou excluir dados relacionados.

01

Schemas e tipos de dados

O tipo correto expressa o domínio, evita ambiguidades e condiciona precisão, armazenamento e operações.

Schema organiza objetos sob um mesmo espaço de nomes

- No MySQL, CREATE SCHEMA é sinônimo de CREATE DATABASE.
- Um schema reúne tabelas, visões, rotinas e outros objetos relacionados.
- A aplicação seleciona o schema ativo com USE nome_do_schema.
- Nomear o schema pelo domínio ajuda a separar contextos e permissões.

Criar e selecionar o schema é o primeiro passo físico

```
CREATE DATABASE IF NOT EXISTS loja_academica  
  CHARACTER SET utf8mb4  
  COLLATE utf8mb4_0900_ai_ci;  
  
USE loja_academica;
```

Trechos didáticos para MySQL 8.4; valide no ambiente da disciplina.

Charset e collation determinam representação e comparação textual

utf8mb4

Representa plenamente Unicode e é a escolha geral recomendável.

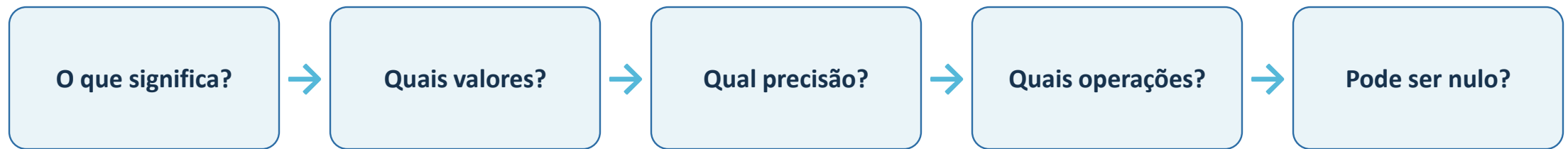
Collation

Define regras de comparação, ordenação, acentos e sensibilidade a maiúsculas.

Consistência

Aplicação, conexão, schema e colunas devem concordar para evitar conversões.

Escolher um tipo é combinar semântica, faixa, precisão e operações



INT representa quantidades e identificadores inteiros

Adequado

Contadores, quantidades, códigos internos e identificadores substitutos.

Não adequado

CPF, CEP e telefone: apesar dos dígitos, não são números para cálculo.

Exemplo

id_produto INT; quantidade INT; ano SMALLINT.

DECIMAL preserva precisão exata em valores financeiros

```
preco_unitario DECIMAL(10,2) NOT NULL
```

```
-- 10 = total de algarismos
```

```
-- 2 = algarismos após a vírgula
```

```
-- maior valor positivo: 99.999.999,99
```

Trechos didáticos para MySQL 8.4; valide no ambiente da disciplina.

CHAR reserva comprimento fixo para valores uniformes

Bom uso

UF CHAR(2), códigos com tamanho realmente invariável.

Armazenamento

Valores são tratados em largura definida; espaços finais têm comportamento específico.

Risco

Usar CHAR para nomes desperdiça espaço e mascara variação natural.

VARCHAR modela textos curtos de comprimento variável

Bom uso

Nome, e-mail, título, endereço e descrições curtas.

Limite

VARCHAR(n) limita caracteres, sujeito ao tamanho máximo da linha e charset.

Domínio

O valor de n deve vir da regra e do uso, não de um número arbitrário.

DATE registra uma data civil sem horário

Representação

Formato lógico AAAA-MM-DD.

Adequado

Nascimento, vencimento, feriado, data de matrícula.

Pergunta-chave

O fuso horário importa? Se não há hora, normalmente não.

DATETIME registra data e hora como valor de calendário

Representação

AAAA-MM-DD hh:mm:ss, com frações
opcionais.

Adequado

Agendamento, emissão, ocorrência e
data_hora do pedido.

Projeto

Defina política de fuso na aplicação;
DATETIME não a resolve sozinho.

BOOLEAN no MySQL é uma forma de TINYINT(1)

ativo BOOLEAN NOT NULL DEFAULT TRUE

-- TRUE é interpretado como 1

-- FALSE é interpretado como 0

-- Se apenas 0 e 1 forem válidos, explicita:
CHECK (ativo IN (0,1))

Trechos didáticos para MySQL 8.4; valide no ambiente da disciplina.

TEXT guarda conteúdo extenso quando VARCHAR deixa de ser natural

Adequado

Observações, corpo de mensagem, descrição livre e conteúdo longo.

Cautela

Não escolha TEXT apenas para evitar decidir um limite.

Índices e FKs

TEXT tem restrições de indexação; não é tipo para chaves estrangeiras.

A semântica orienta a família de tipos

Necessidade	Tipo inicial	Exemplo
Inteiro	INT	quantidade
Decimal exato	DECIMAL(p,s)	preço
Código fixo	CHAR(n)	UF
Texto curto variável	VARCHAR(n)	nome
Data	DATE	nascimento
Instante civil	DATETIME	data_hora
Verdadeiro/falso	BOOLEAN	ativo

O modelo lógico do comércio eletrônico ganha tipos físicos

Atributo lógico	Declaração física	Justificativa
id_cliente	INT	Identificador substituto
email	VARCHAR(254)	Texto curto e variável
preco_atual	DECIMAL(10,2)	Precisão financeira
data_hora	DATETIME	Data e hora do pedido
status	VARCHAR(20)	Vocabulário controlado
observacao	TEXT	Conteúdo livre opcional

02

Constraints e identidade

Tipos delimitam a forma dos valores; constraints tornam explícitas as regras que toda linha deverá respeitar.

CREATE TABLE combina colunas, tipos e restrições

```
CREATE TABLE cliente (  
  id_cliente INT NOT NULL AUTO_INCREMENT,  
  nome VARCHAR(120) NOT NULL,  
  email VARCHAR(254) NOT NULL,  
  PRIMARY KEY (id_cliente),  
  CONSTRAINT uq_cliente_email UNIQUE (email)  
) ENGINE = InnoDB;
```

Trechos didáticos para MySQL 8.4; valide no ambiente da disciplina.

PRIMARY KEY garante identidade única e não nula

Única

Duas linhas não podem compartilhar a mesma chave primária.

Não nula

Nenhum componente da PK pode ser NULL.

Uma por tabela

Pode ser simples ou composta, mas há uma única PK declarada.

AUTO_INCREMENT gera o próximo valor de uma coluna inteira

Uso típico

Chave substituta inteira em tabelas como CLIENTE e PEDIDO.

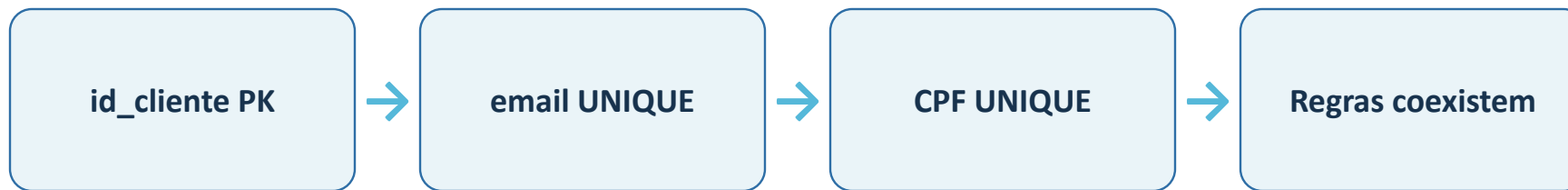
Não significa

Não garante sequência sem lacunas nem carrega significado de negócio.

Prática

A aplicação normalmente omite a coluna no INSERT e recupera o valor gerado.

Chave substituta não elimina as chaves naturais do domínio

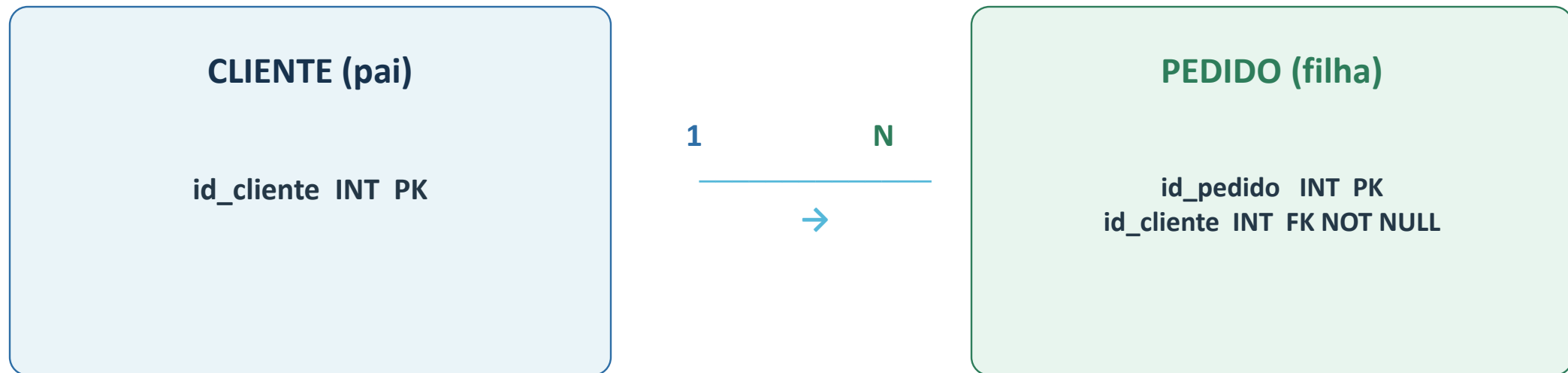


FOREIGN KEY exige correspondência com uma chave referenciada

```
CONSTRAINT fk_pedido_cliente  
  FOREIGN KEY (id_cliente)  
  REFERENCES cliente (id_cliente)  
  ON DELETE RESTRICT  
  ON UPDATE RESTRICT
```

Trechos didáticos para MySQL 8.4; valide no ambiente da disciplina.

A tabela filha carrega a referência para a tabela pai



A FK está no lado N

UNIQUE protege identificadores alternativos e relações 1:1

Identificador alternativo

email e matrícula podem ser exclusivos sem serem a PK.

Relacionamento 1:1

FK + UNIQUE impede múltiplos dependentes para o mesmo pai.

NULL

No MySQL, UNIQUE admite múltiplos NULL; combine com NOT NULL quando necessário.

NOT NULL transforma participação obrigatória em regra física

Sem NOT NULL

A coluna pode omitir o valor, se outras regras permitirem.

Com NOT NULL

INSERT e UPDATE devem fornecer um valor não nulo.

FK obrigatória

NOT NULL + FOREIGN KEY expressam vínculo existente e obrigatório.

DEFAULT fornece um valor quando a coluna é omitida

```
status VARCHAR(20) NOT NULL DEFAULT 'ABERTO',  
criado_em DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,  
ativo BOOLEAN NOT NULL DEFAULT TRUE
```

-- DEFAULT não corrige valor inválido explicitamente informado.

Trechos didáticos para MySQL 8.4; valide no ambiente da disciplina.

CHECK rejeita linhas cujo predicado não seja verdadeiro

```
quantidade INT NOT NULL,  
preco_praticado DECIMAL(10,2) NOT NULL,  
status VARCHAR(20) NOT NULL,  
CONSTRAINT ck_item_quantidade CHECK (quantidade > 0),  
CONSTRAINT ck_item_preco CHECK (preco_praticado >= 0),  
CONSTRAINT ck_pedido_status CHECK (status IN ('ABERTO','PAGO','CANCELADO'))
```

Trechos didáticos para MySQL 8.4; valide no ambiente da disciplina.

Constraints de coluna e de tabela diferem pelo alcance

Coluna

Regra aparece junto à coluna e normalmente envolve apenas aquele valor.

Tabela

Permite nome explícito, chave composta e comparação entre várias colunas.

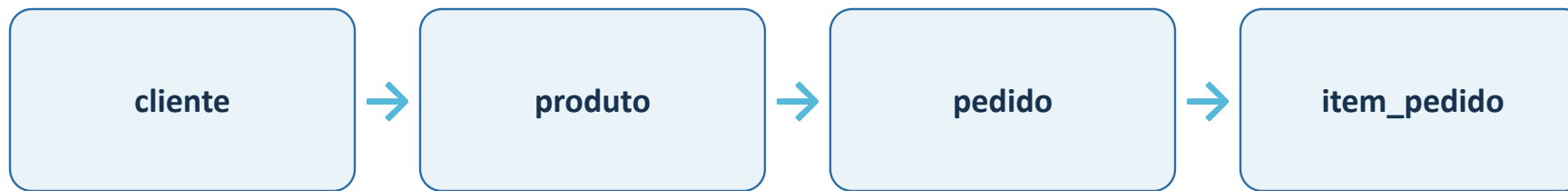
Exemplo

```
CHECK (data_fim IS NULL OR data_fim  
      >= data_inicio).
```

Cada constraint responde a uma pergunta diferente

Constraint	Pergunta protegida	Exemplo
PRIMARY KEY	Quem é esta linha?	id_pedido
FOREIGN KEY	O vínculo existe?	id_cliente
UNIQUE	Este valor pode repetir?	email
NOT NULL	O valor é obrigatório?	nome
DEFAULT	O que assumir se omitido?	status
CHECK	O valor satisfaz a regra?	quantidade > 0

Pais devem existir antes das FKs que os referenciam



CLIENTE: identidade substituta e e-mail alternativo

```
CREATE TABLE cliente (  
  id_cliente INT NOT NULL AUTO_INCREMENT,  
  nome VARCHAR(120) NOT NULL,  
  email VARCHAR(254) NOT NULL,  
  ativo BOOLEAN NOT NULL DEFAULT TRUE,  
  CONSTRAINT pk_cliente PRIMARY KEY (id_cliente),  
  CONSTRAINT uq_cliente_email UNIQUE (email),  
  CONSTRAINT ck_cliente_ativo CHECK (ativo IN (0,1))  
) ENGINE = InnoDB;
```

Trechos didáticos para MySQL 8.4; valide no ambiente da disciplina.

PRODUTO: preço exato e valores semanticamente válidos

```
CREATE TABLE produto (  
  id_produto INT NOT NULL AUTO_INCREMENT,  
  nome VARCHAR(160) NOT NULL,  
  preco_atual DECIMAL(10,2) NOT NULL,  
  estoque INT NOT NULL DEFAULT 0,  
  CONSTRAINT pk_produto PRIMARY KEY (id_produto),  
  CONSTRAINT ck_produto_preco CHECK (preco_atual >= 0),  
  CONSTRAINT ck_produto_estoque CHECK (estoque >= 0)  
) ENGINE = InnoDB;
```

Trechos didáticos para MySQL 8.4; valide no ambiente da disciplina.

PEDIDO: vínculo obrigatório e estado controlado

```
CREATE TABLE pedido (  
  id_pedido INT NOT NULL AUTO_INCREMENT,  
  data_hora DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,  
  status VARCHAR(20) NOT NULL DEFAULT 'ABERTO',  
  id_cliente INT NOT NULL,  
  CONSTRAINT pk_pedido PRIMARY KEY (id_pedido),  
  CONSTRAINT ck_pedido_status CHECK (status IN ('ABERTO','PAGO','CANCELADO')),  
  CONSTRAINT fk_pedido_cliente FOREIGN KEY (id_cliente)  
    REFERENCES cliente(id_cliente) ON DELETE RESTRICT ON UPDATE RESTRICT  
) ENGINE = InnoDB;
```

Trechos didáticos para MySQL 8.4; valide no ambiente da disciplina.

ITEM_PEDIDO: PK composta preserva um produto por linha do pedido

```
CREATE TABLE item_pedido (  
  id_pedido INT NOT NULL,  
  id_produto INT NOT NULL,  
  quantidade INT NOT NULL,  
  preco_praticado DECIMAL(10,2) NOT NULL,  
  CONSTRAINT pk_item PRIMARY KEY (id_pedido, id_produto),  
  CONSTRAINT ck_item_qtd CHECK (quantidade > 0),  
  CONSTRAINT ck_item_preco CHECK (preco_praticado >= 0),  
  CONSTRAINT fk_item_pedido FOREIGN KEY (id_pedido) REFERENCES pedido(id_pedido) ON DELETE CASCADE,  
  CONSTRAINT fk_item_produto FOREIGN KEY (id_produto) REFERENCES produto(id_produto) ON DELETE RESTRICT  
) ENGINE = InnoDB;
```

Trechos didáticos para MySQL 8.4; valide no ambiente da disciplina.

03

Integridade, ações referenciais e anomalias

Uma estrutura física adequada não apenas armazena dados: ela impede estados inválidos e torna explícito o ciclo de vida dos vínculos.

Integridade atua em três níveis complementares

Domínio

Tipo, nulidade, faixa, formato e predicados permitidos.

Entidade

Cada linha possui identidade única, estável e não nula.

Referencial

Toda FK aponta para uma chave existente ou admite ausência legítima.

O MySQL valida a operação antes de aceitar um novo estado



ON DELETE decide o destino dos filhos quando o pai é excluído

RESTRICT / NO ACTION

Impede excluir o pai enquanto existirem filhos.

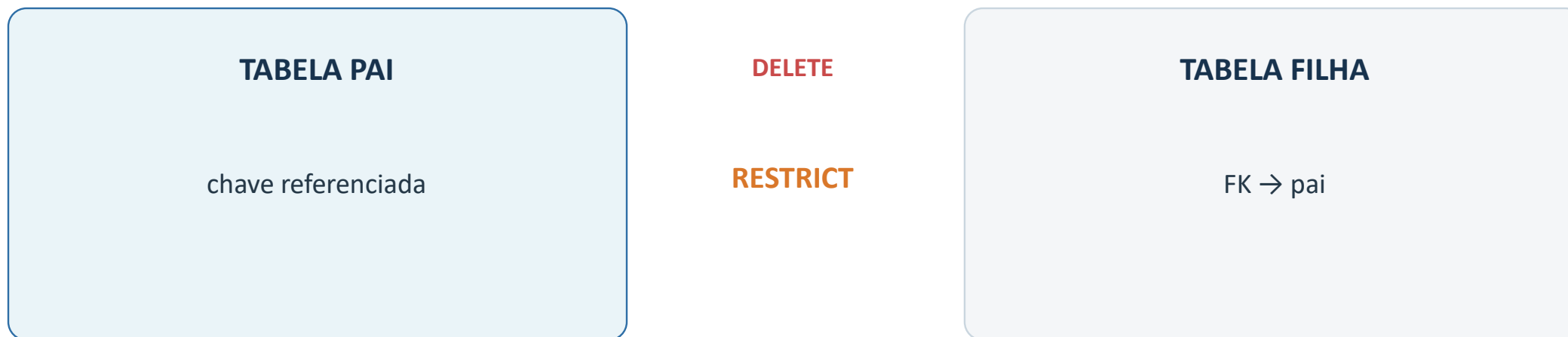
CASCADE

Exclui automaticamente os filhos correspondentes.

SET NULL

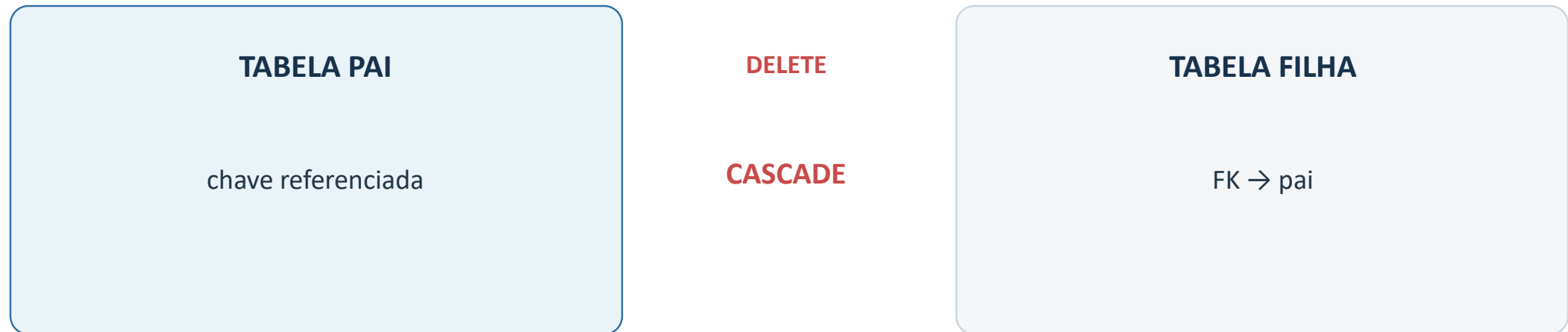
Mantém o filho, mas remove o vínculo; a FK precisa aceitar NULL.

RESTRICT protege históricos que não devem desaparecer por acidente



Excluir CLIENTE é bloqueado enquanto houver PEDIDOS. O usuário deve tratar os dependentes conscientemente.

CASCADE é adequado quando o filho não tem sentido sem o pai



Excluir PEDIDO exclui seus ITEM_PEDIDO. A linha do pedido é parte composicional do pedido.

SET NULL preserva o filho e registra que o vínculo deixou de existir



Ao excluir um responsável, a tarefa permanece sem responsável. A coluna FK deve aceitar NULL.

A ação correta depende da semântica e do ciclo de vida

Ação	Mantém filho?	Uso coerente
RESTRICT	Sim, sem alteração	Histórico impede exclusão do pai
CASCADE	Não	Filho é parte inseparável do pai
SET NULL	Sim, sem vínculo	Relacionamento é opcional
NO ACTION	Sim; operação rejeitada	Equivale a RESTRICT no InnoDB

ON UPDATE trata alterações da chave referenciada

RESTRICT

Bloqueia alteração da chave pai com dependentes.

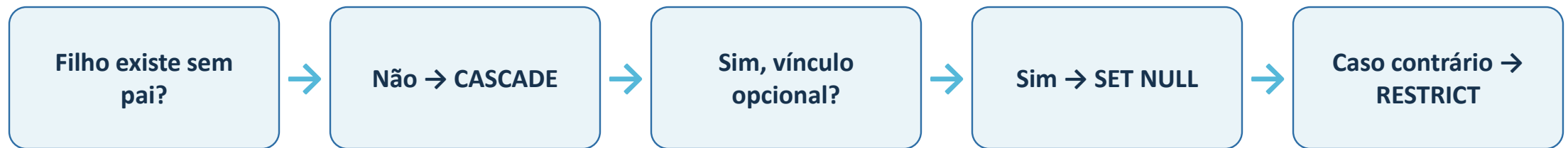
CASCADE

Propaga o novo valor às FKs filhas.

Decisão prática

Chaves substitutas deveriam ser estáveis; atualizar PK costuma ser excepcional.

Escolha políticas referenciais respondendo ao ciclo de vida



Redundância é a repetição do mesmo fato em lugares independentes

Exemplo

Guardar nome_cliente em CLIENTE e novamente em cada PEDIDO.

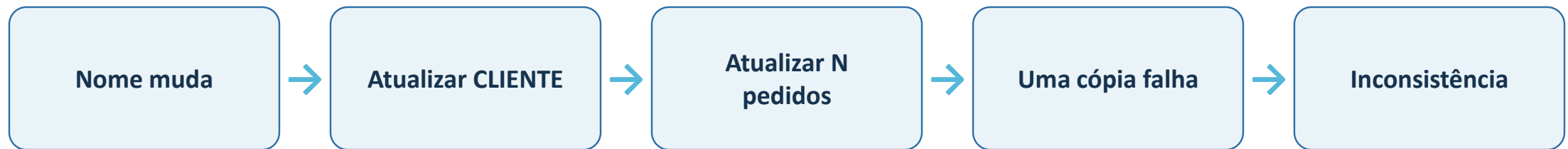
Risco

As cópias podem divergir após alterações parciais.

Exceção consciente

Desnormalização exige motivo, regra de sincronização e mensuração.

Uma mudança exige múltiplas escritas e pode produzir versões conflitantes



Misturar fatos impede registrar algo sem inventar outro fato

Tabela misturada

CURSO_TURMA contém dados do curso e da turma.

Problema

Não se registra um novo curso antes de existir uma turma.

Correção

Separar CURSO e TURMA e relacioná-los por FK.

Excluir o último fato dependente pode apagar informação independente

Tabela misturada

Dados do professor repetidos em cada oferta.

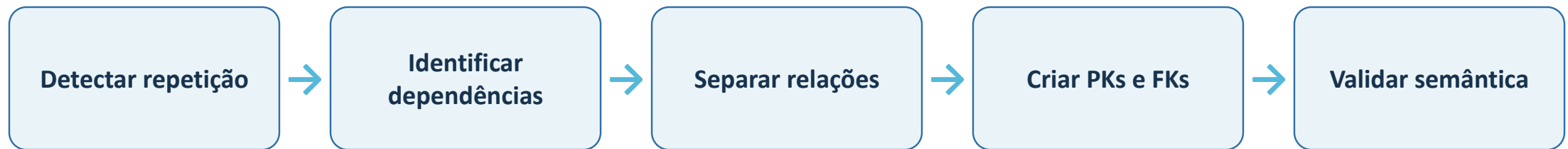
Problema

Excluir a última oferta remove também o único registro do professor.

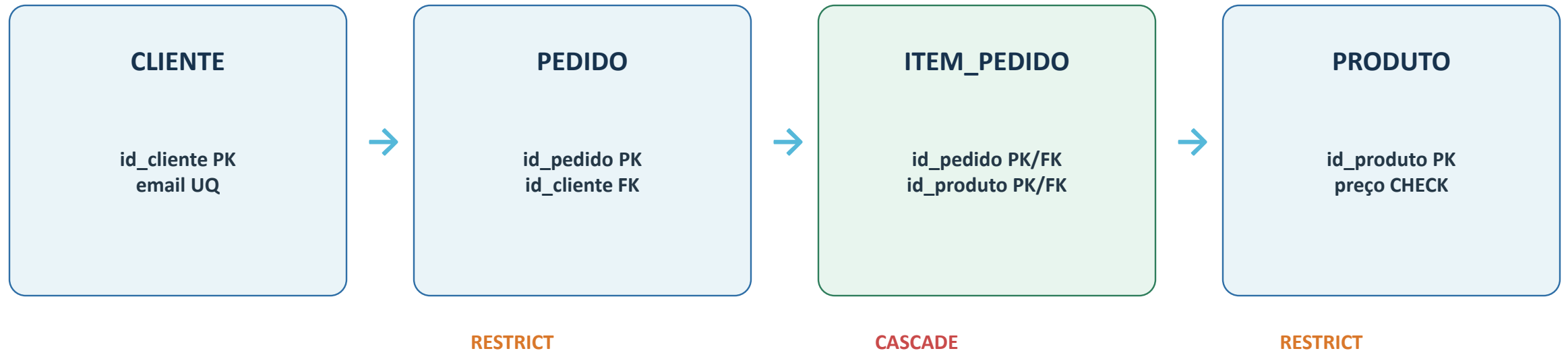
Correção

Entidades independentes precisam de relações próprias.

Normalização reduz anomalias ao separar fatos com dependências distintas



O comércio eletrônico expressa tipo, identidade, vínculo e ciclo de vida



Violações planejadas comprovam que o projeto protege o domínio

Teste	Regra esperada	Resultado
Preço negativo	CHECK	Rejeitar
Email duplicado	UNIQUE	Rejeitar
Pedido sem cliente	FK / NOT NULL	Rejeitar
Excluir cliente com pedido	RESTRICT	Rejeitar
Excluir pedido	CASCADE em ITEM	Excluir itens
Status omitido	DEFAULT	Assumir ABERTO

DDL sintaticamente válido ainda pode implementar regras erradas

- Usar VARCHAR para tudo e perder semântica, precisão e validação.
- Confundir DEFAULT com obrigatoriedade ou correção automática.
- Declarar FK sem NOT NULL quando o relacionamento é obrigatório.
- Usar CASCADE por conveniência, sem analisar o ciclo de vida.
- Duplicar atributos descritivos em tabelas filhas e criar anomalias.
- Confiar somente na aplicação para regras que o banco pode garantir.

04

Exercícios resolvidos e propostos

Primeiro, acompanhe quatro resoluções comentadas; depois, aplique as decisões em cinco níveis e em um desafio integrador.

Selecione tipos para ALUNO e justifique cada decisão

Atributo	Solução	Justificativa
id_aluno	INT AUTO_INCREMENT	Identidade interna
matricula	CHAR(10)	Código fixo; preserva zeros
nome	VARCHAR(120)	Texto curto variável
email	VARCHAR(254)	Identificador alternativo
data_nascimento	DATE	Data sem horário
bolsista	BOOLEAN	Estado lógico

Transforme as regras de MATRÍCULA em constraints físicas

```
CREATE TABLE matricula (  
  id_matricula INT NOT NULL AUTO_INCREMENT,  
  id_aluno INT NOT NULL,  
  id_turma INT NOT NULL,  
  situacao VARCHAR(15) NOT NULL DEFAULT 'ATIVA',  
  PRIMARY KEY (id_matricula),  
  UNIQUE (id_aluno, id_turma),  
  CHECK (situacao IN ('ATIVA','TRANCADA','CONCLUÍDA')),  
  FOREIGN KEY (id_aluno) REFERENCES aluno(id_aluno),  
  FOREIGN KEY (id_turma) REFERENCES turma(id_turma)  
) ENGINE=InnoDB;
```

Trechos didáticos para MySQL 8.4; valide no ambiente da disciplina.

Justifique políticas diferentes para PEDIDO e ITEM_PEDIDO

Vínculo	Política	Raciocínio
CLIENTE → PEDIDO	ON DELETE RESTRICT	Pedido é histórico; não deve desaparecer com o cliente
PEDIDO → ITEM_PEDIDO	ON DELETE CASCADE	Item não possui sentido sem o pedido
PRODUTO → ITEM_PEDIDO	ON DELETE RESTRICT	Produto vendido participa do histórico
PKs substitutas	ON UPDATE RESTRICT	Identificadores devem permanecer estáveis

Diagnostique a tabela PEDIDO_COMPLETO e proponha a decomposição

Sintoma	Anomalia	Correção
nome_cliente repetido em cada item	Atualização	Manter nome somente em CLIENTE
produto só existe após uma venda	Inserção	Criar PRODUTO independente
excluir último item apaga produto	Exclusão	Separar PRODUTO de ITEM_PEDIDO
preço praticado muda com preço atual	Histórica	Guardar preço_praticado no vínculo

Escolha tipos para um sistema acadêmico

- Modele ALUNO(id, matrícula, nome, e-mail, nascimento, coeficiente, bolsista e observações).
- Escolha um tipo MySQL para cada atributo.
- Defina comprimentos e precisão com justificativas.
- Identifique códigos que não devem ser tratados como números.
- Diferencie dado ausente, inaplicável e obrigatório.

Declare a tabela RESERVA com constraints coerentes

- Use id_reserva, id_sala, id_responsavel, inicio, fim e situacao.
- Defina PK, FKs, NOT NULL e DEFAULT.
- Imponha CHECK para fim posterior a inicio.
- Controle o vocabulário de situacao.
- Explique por que UNIQUE(id_sala, inicio) não detecta toda sobreposição temporal.

Escolha ON DELETE e ON UPDATE para três vínculos

- DEPARTAMENTO → EMPREGADO: o empregado pode permanecer sem departamento?
- PROJETO → TAREFA: a tarefa possui existência independente?
- AUTOR → LIVRO_AUTOR: o contrato histórico pode ser apagado?
- Para cada vínculo, escolha RESTRICT, CASCADE ou SET NULL.
- Justifique pela obrigatoriedade, história e ciclo de vida — não por conveniência.

Projete violações que demonstrem a eficácia das constraints

- Crie um INSERT que viole PRIMARY KEY.
- Crie um INSERT que viole UNIQUE e outro que viole CHECK.
- Tente inserir uma FK órfã.
- Teste uma exclusão bloqueada por RESTRICT.
- Teste uma exclusão propagada por CASCADE e descreva o estado final esperado.

Encontre redundâncias e anomalias em uma relação não normalizada

- Analise VENDA(id_venda, data, id_cliente, nome_cliente, id_produto, nome_produto, quantidade).
- Identifique os fatos distintos misturados.
- Demonstre uma anomalia de atualização, uma de inserção e uma de exclusão.
- Decomponha a relação em tabelas com PKs e FKs.
- Indique qual atributo histórico deve permanecer no item da venda.

Implemente o modelo físico de uma clínica

- Crie schema e tabelas PACIENTE, MÉDICO, CONSULTA, PROCEDIMENTO e CONSULTA_PROCEDIMENTO.
- Escolha tipos para identificadores, textos, data_hora, valores e estados.
- Declare PKs, FKs, UNIQUE, NOT NULL, DEFAULT e CHECK.
- Defina e justifique ON DELETE e ON UPDATE para cada vínculo.
- Planeje ao menos seis testes de violação e preveja cada resultado.
- Revise redundâncias, dados históricos e possíveis anomalias.

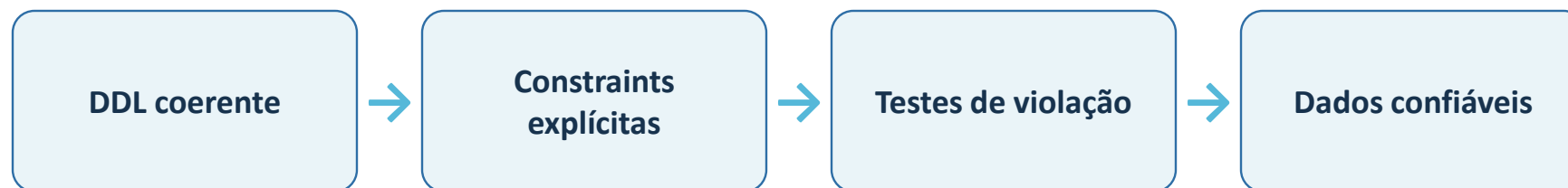
O essencial da T05 em cinco ideias

- Tipos físicos devem expressar domínio, precisão e operações esperadas.
- PK identifica; UNIQUE protege alternativas; FK preserva vínculos.
- NOT NULL, DEFAULT e CHECK respondem a regras diferentes.
- ON DELETE e ON UPDATE traduzem decisões de ciclo de vida.
- Redundância não controlada cria anomalias de atualização, inserção e exclusão.

Recuperação ativa: responda sem consultar os slides

- Por que CPF deve ser CHAR/VARCHAR e preço deve ser DECIMAL?
- Qual diferença existe entre PRIMARY KEY, UNIQUE e NOT NULL?
- Quando ON DELETE CASCADE é coerente — e quando é perigoso?
- Por que SET NULL exige uma FK anulável?
- Como uma redundância produz anomalia de atualização?
- Que violações você testaria antes de considerar o schema confiável?

O modelo físico transforma regras em comportamento verificável



Até a próxima!